

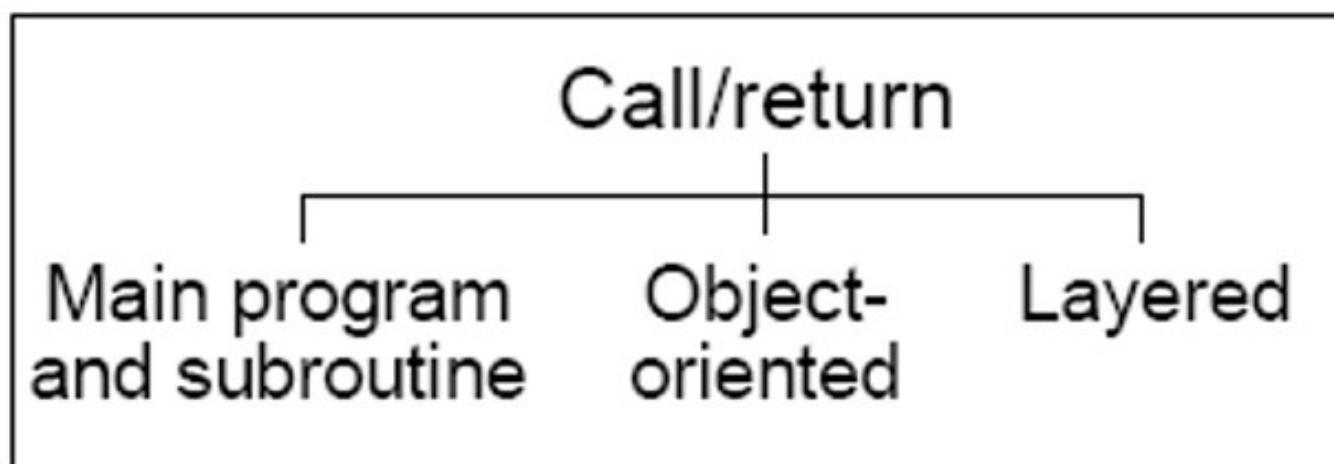


软件体系结构

第8章 主程序-子程序与层次风格

赵庆玲

南京理工大学计算机科学与工程学院

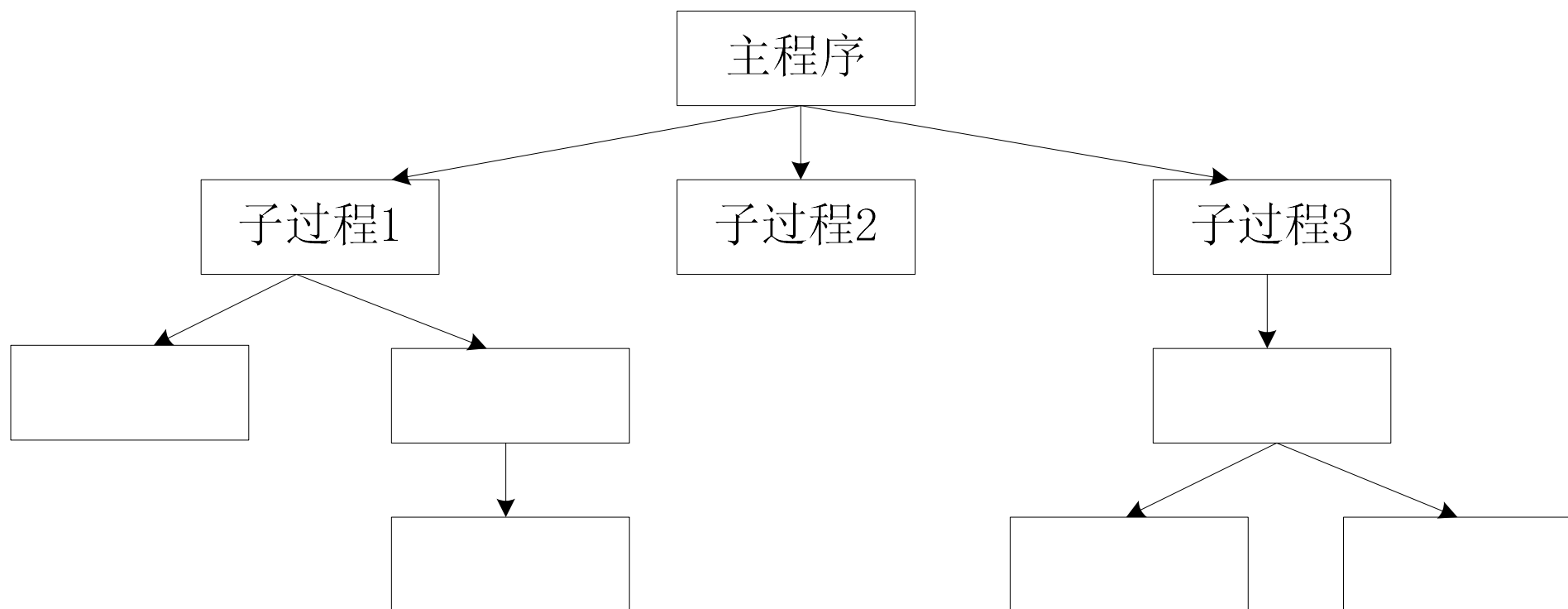


- 20世纪70年代，随结构化程序设计方式出现而出现
- 该架构风格从功能的观点设计系统，通过逐步分解和逐步细化得到系统架构。



主程序/子程序风格

- 其中，组件为主程序和子程序，连接件为调用-返回机制，拓扑结构为层次化结构。

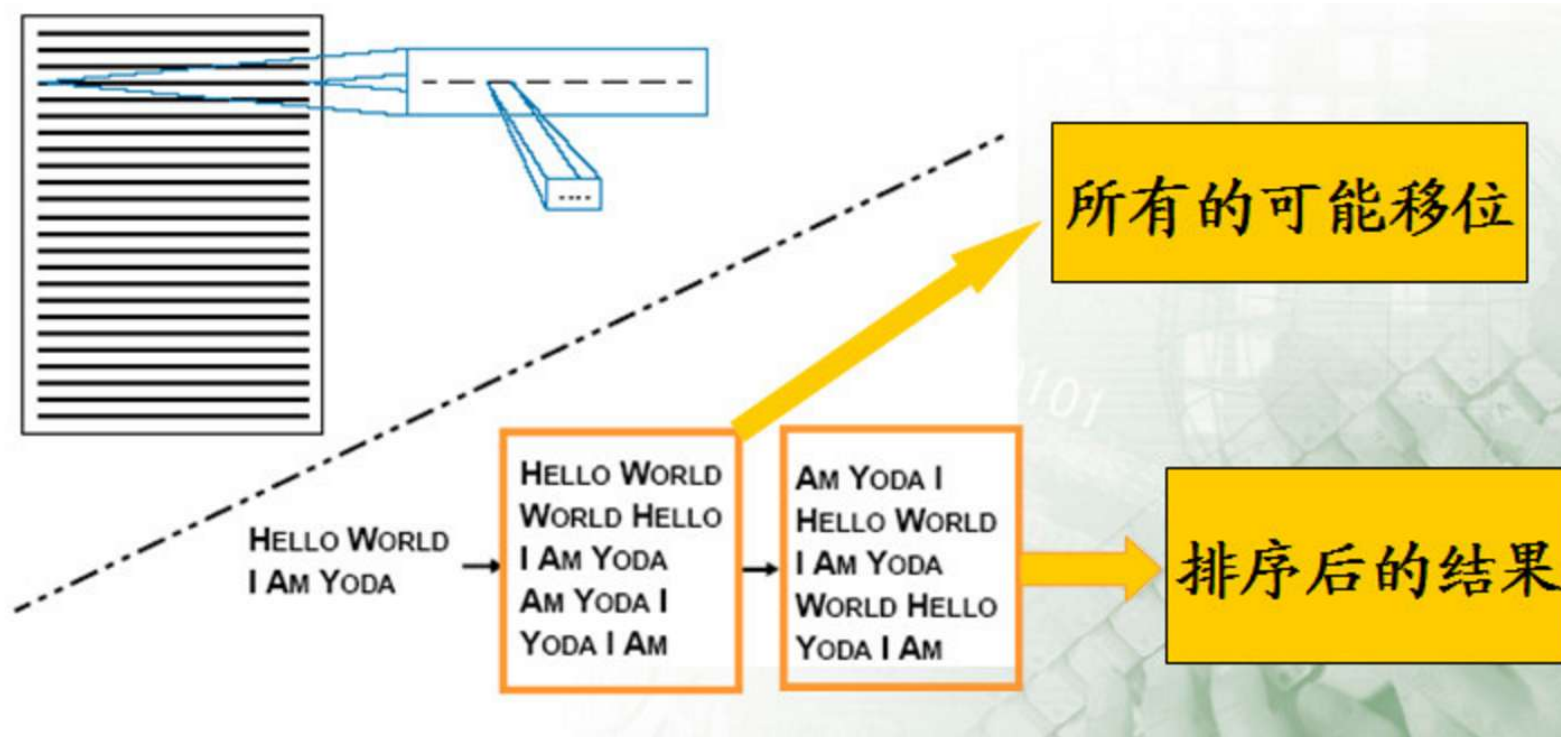


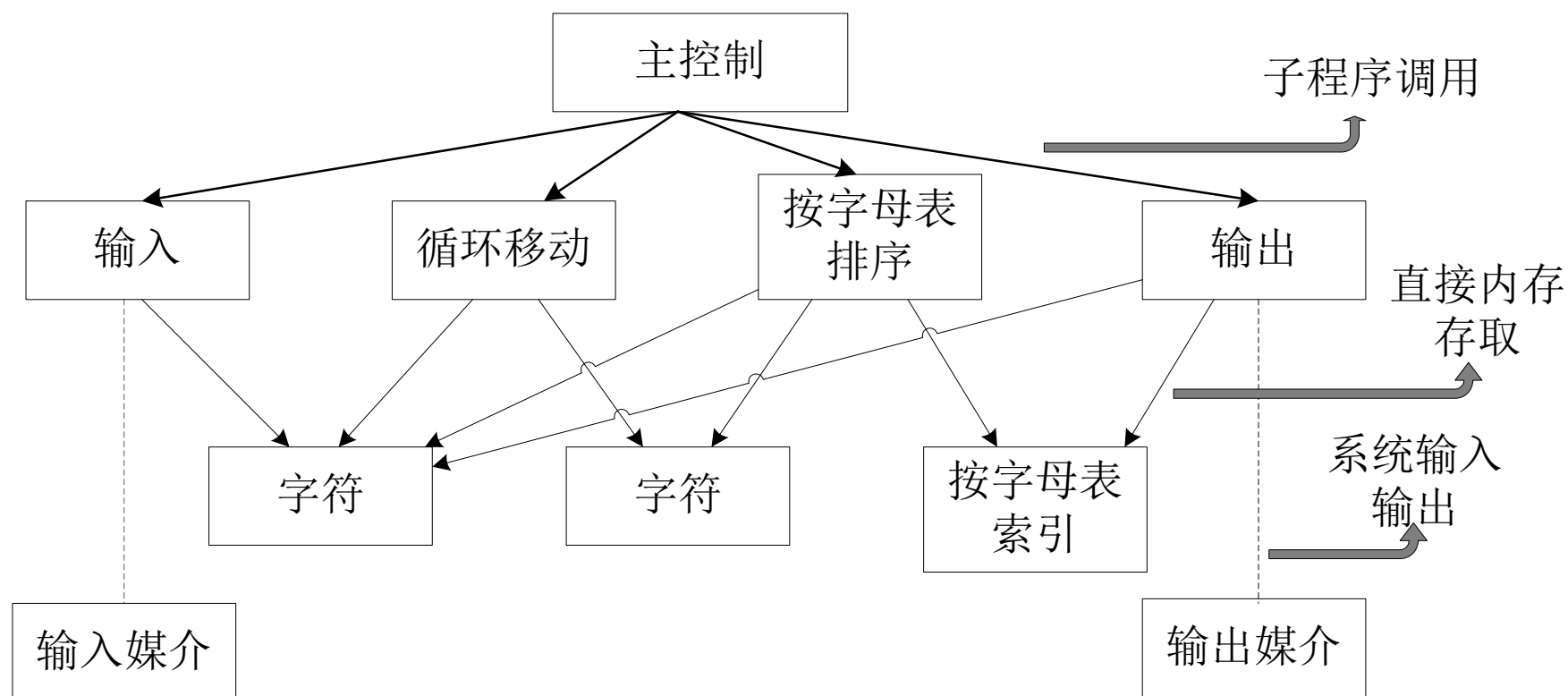
- 优点
 - （1）具有很高的数据访问效率，因为计算共享同一个存储区。
 - （2）不同的计算功能被划分在不同的模块中。
- 缺点：该方案在处理变更的能力上有许多严重的缺陷：
 - （1）对数据存储格式的变化将会影响几乎所有的模块。
 - （2）对处理流程的改变与系统功能的增强也很难适应，依赖于控制模块内部的调用次序。
 - （3）这种分解方案难以支持有效的复用。

KWIC问题



- KWIC(Key Word In Context), Parnas (1972)
- KWIC索引系统接受一些行，每行有若干字，每个字由若干字符组成；每行都可以循环移位，亦即重复地把第一个字删除，然后接到行末；KWIC把所有行的各种移位情况按照字母表顺序输出

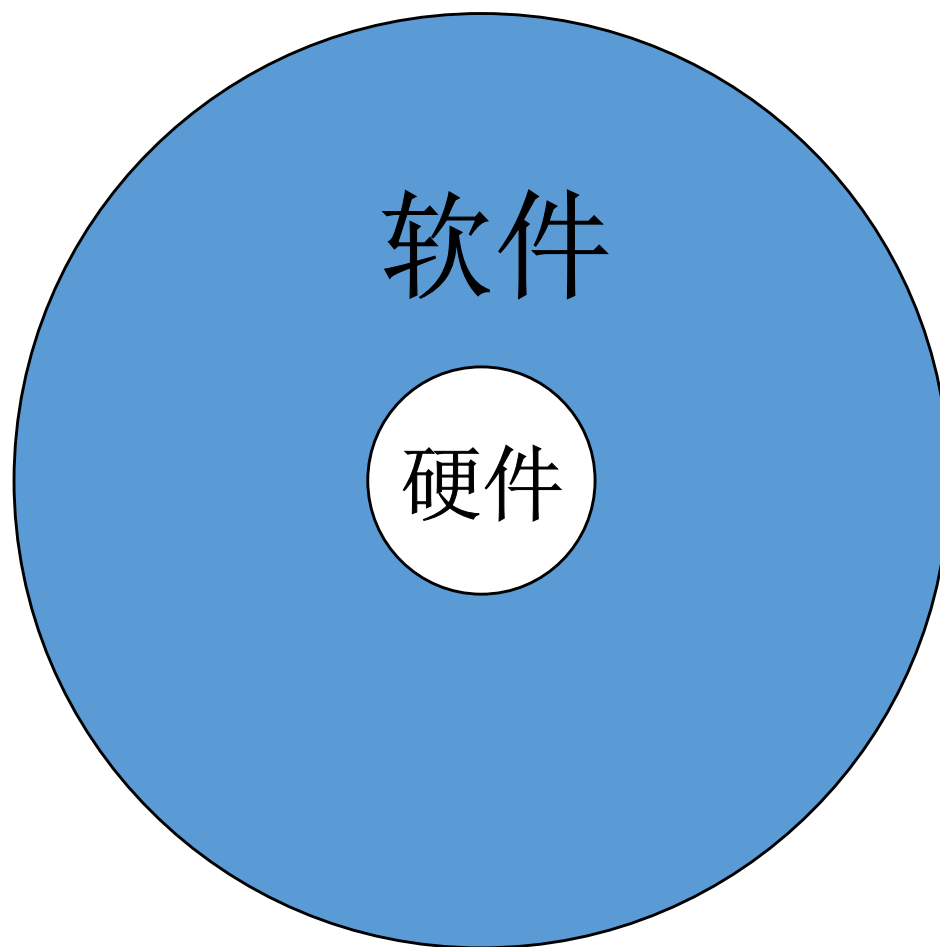




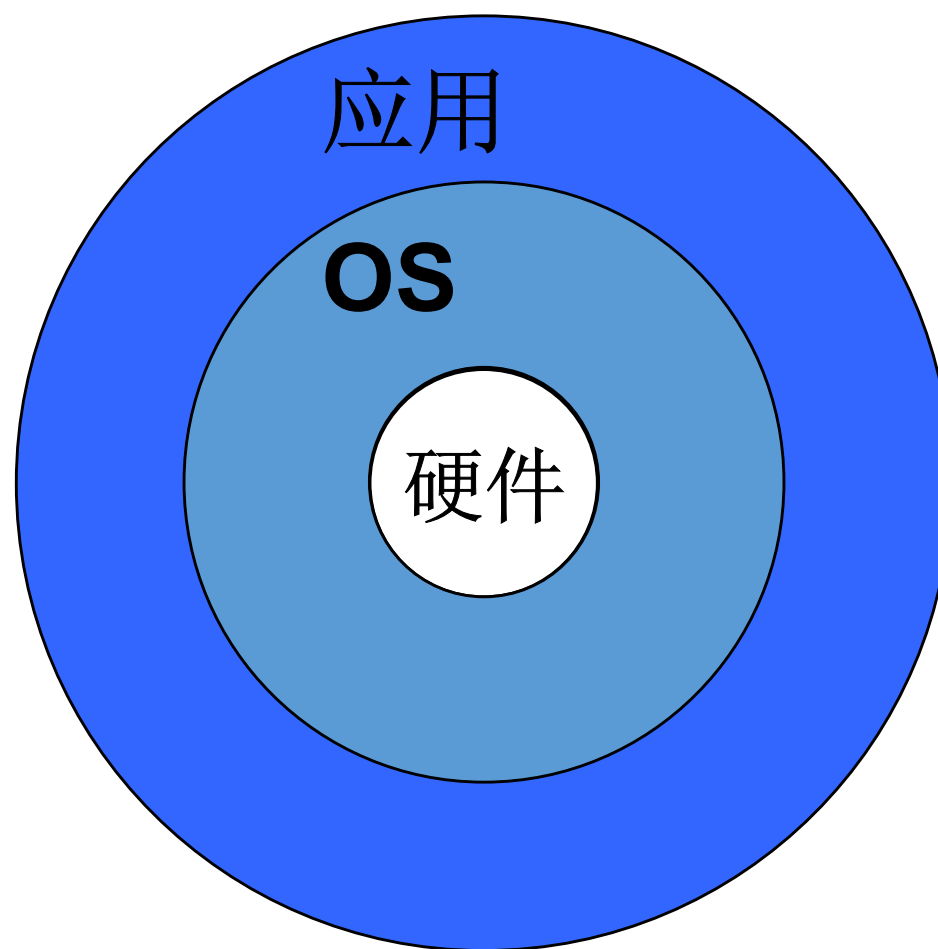
- 基本思想
 - 在分层系统（Layered System）中，系统被组织成若干个层次，每个层次由一系列组件组成；层次之间存在接口，通过接口形成call/return的关系——下层组件向上层组件提供服务，上层组件被看作是下层组件的客户端。
- 层次化早已经成为一种复杂系统设计的普遍性原则。



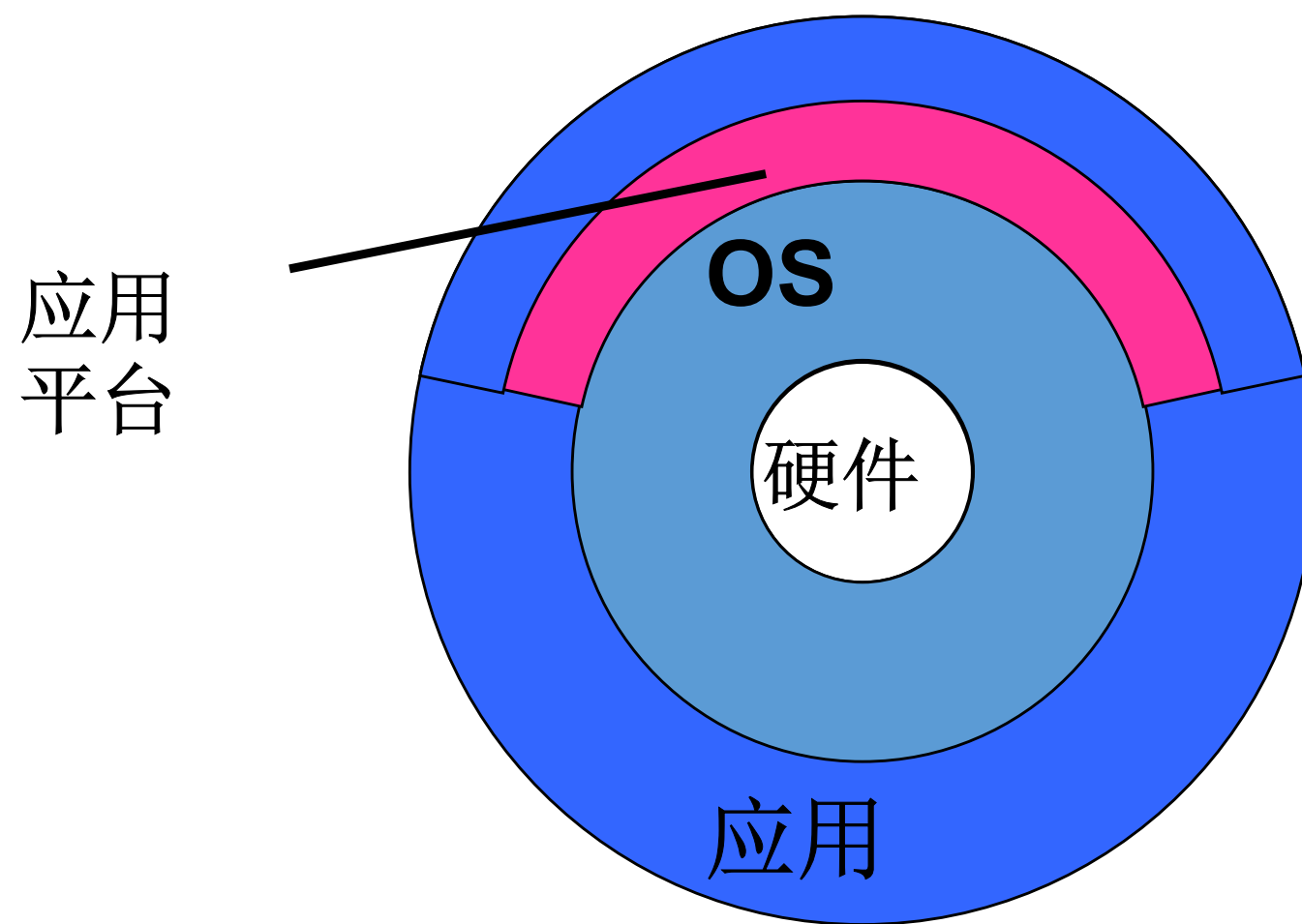
无处不在的层次结构



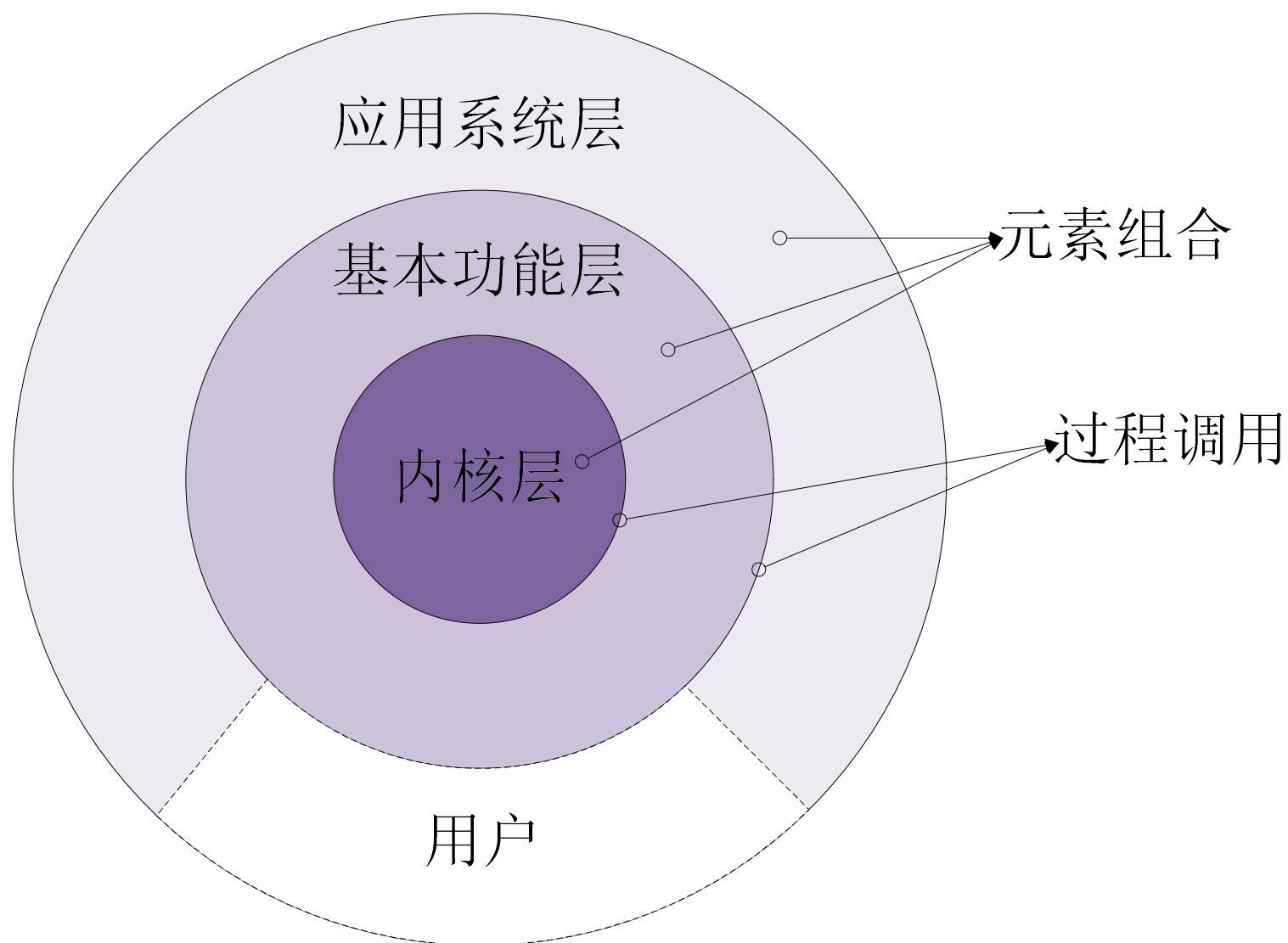
用户



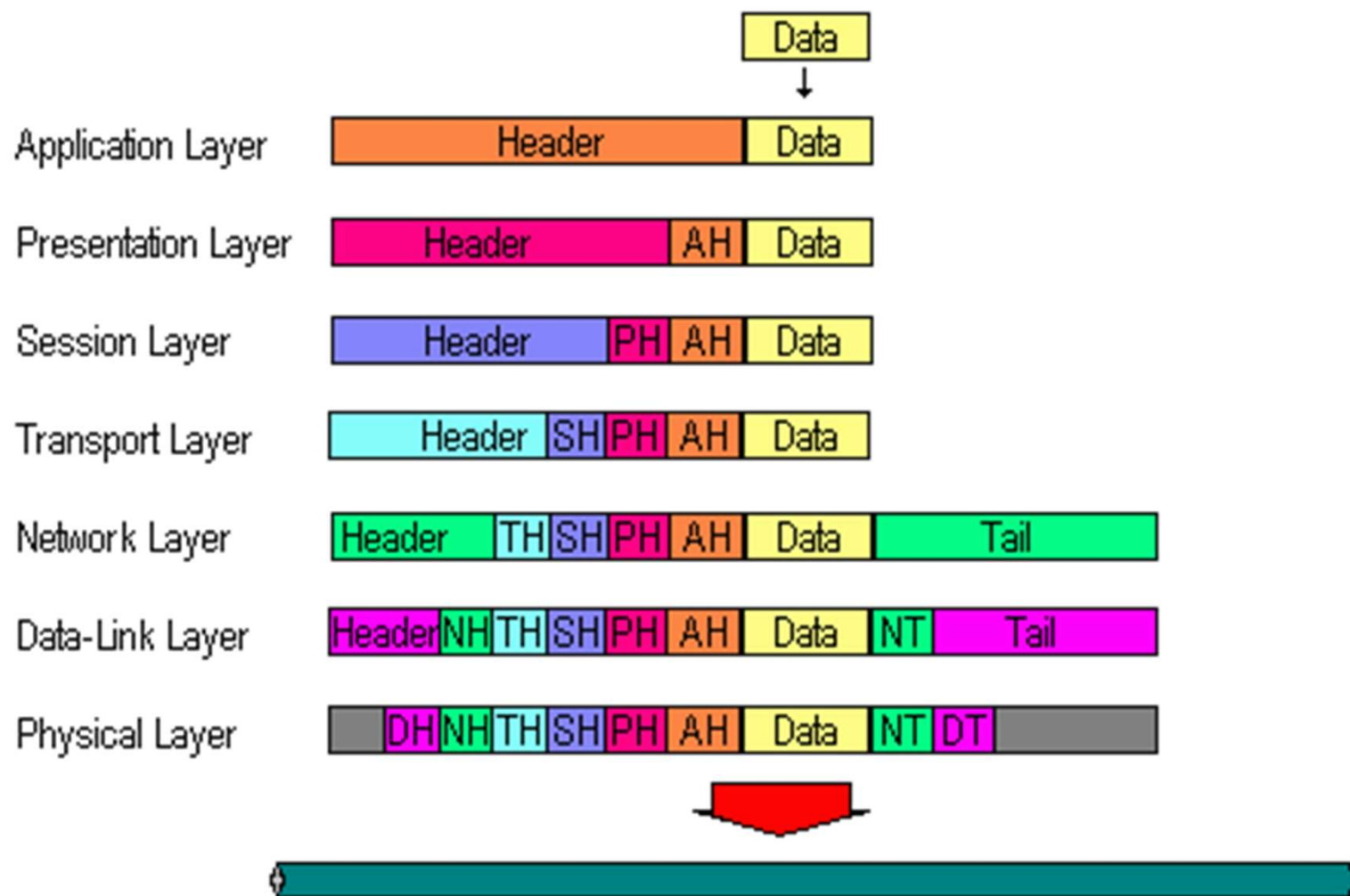
用户



用户

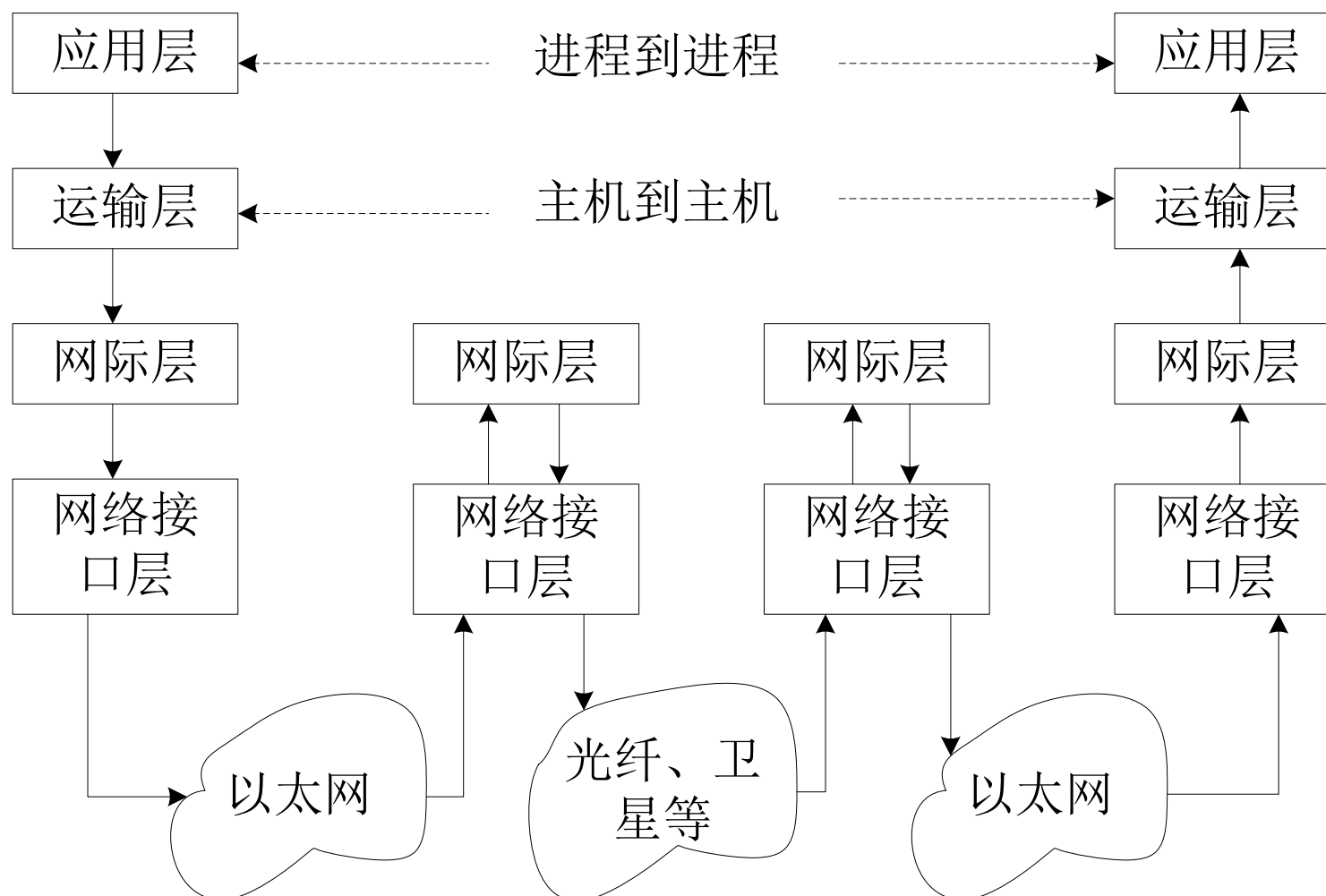


OSI参考模型





TCP/IP分层通信协议



- 每层为上一层提供服务，使用下一层的服务，只能见到与自己邻接的层
 - 适当时候(必不得已的时候)，可以允许一定的越层操作
- 大的问题分解为若干个渐进的小问题，逐步解决，隐藏了很多复杂度
- 修改一层，最多影响两层，而通常只能影响上层。接口稳固，则谁都不影响

- 上层必须知道下层的身份，不能调整层次之间的顺序
- 层层相调，影响性能

- 优点:
 - 支持基于抽象程度递增的系统设计，使设计者可以把一个复杂系统按递增的步骤进行分解
 - 支持功能增强，因为每一层至多和相邻的上下层交互，因此功能的改变最多影响相邻的上下层。
 - 支持重用。只要提供的服务接口定义不变，同一层的不同实现可以交换使用。这样就可以定义一组标准的接口，而允许各种不同的实现方法。

- 缺点

- 并不是每个系统都可以很容易的划分为分层的模式，甚至即使一个系统的逻辑结构是层次化，出于对系统性能的考虑，系统设计师不得不把一些低级或高级的功能综合起来。
- 很难找到一个合适的、正确的层次抽象方法。
 - 例如，实际的通信协议体就很难映射到OSI框架中，因为其中许多协议跨多个层。



Java的I/O

- 层次风格，还是管道风格？

```
DataOutputStream out =  
    new DataOutputStream(  
        new BufferedOutputStream(  
            new FileOutputStream("Data.txt"))));  
out.writeChars("偶是层次风格的吗！");
```

